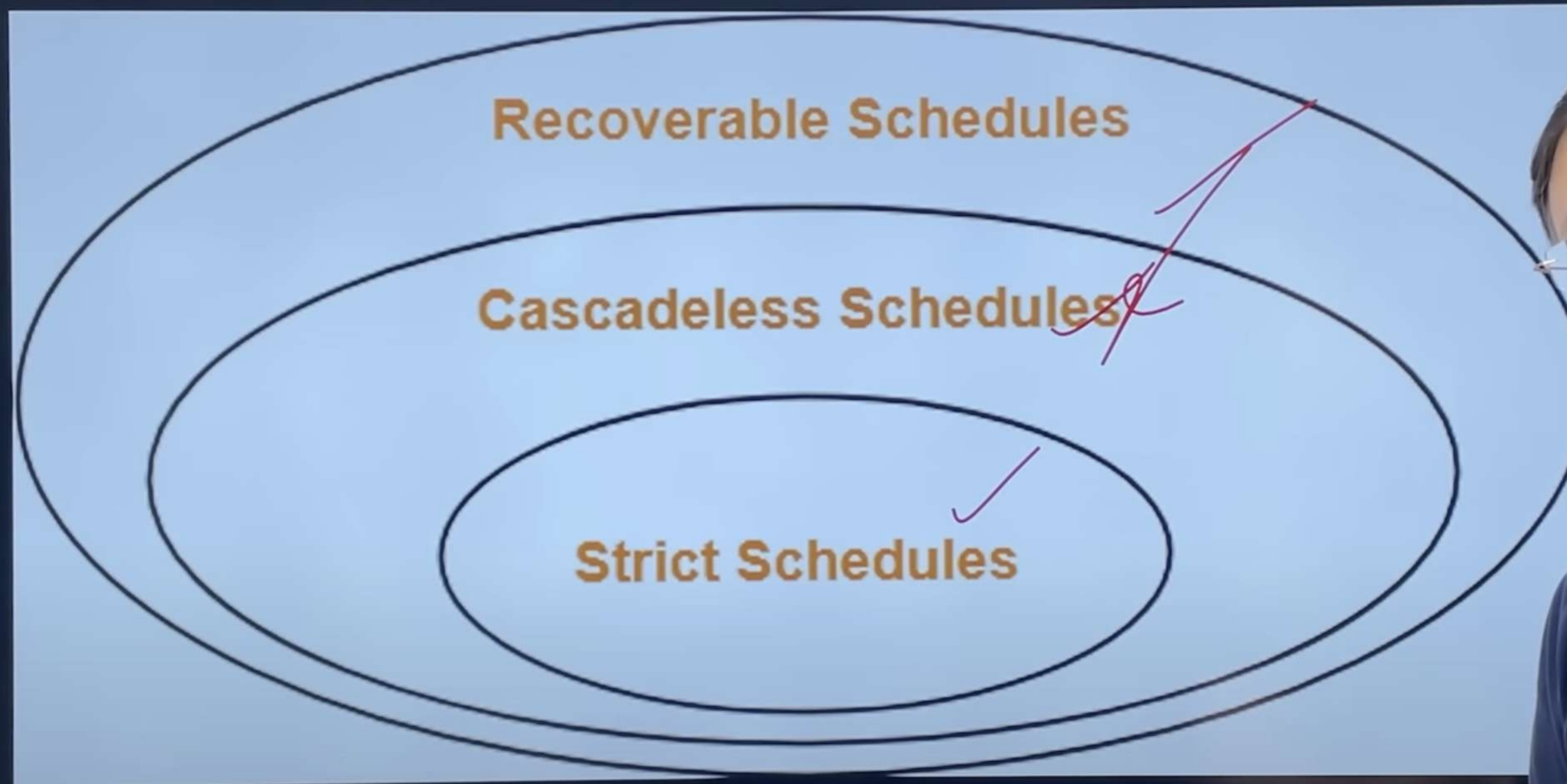




Strict Schedule

- A schedule in which for each pair of transactions T_i and T_j , such that if T_j reads a data item previously written by T_i then the commit or abort of T_i must appear before read and write operation of T_j .

S_1	
T_1	T_2
$R(a)$	
$W(a)$	
	$W(a)$
C	
	$R(a)$
	C

S_2	
T_1	T_2
$R(a)$	
$W(a)$	
C	
	$W(a)$
	$R(a)$
	C

S_3	
T_1	T_2
$R(a)$	
	$R(b)$
$W(a)$	
	$W(b)$
C	

[View chapter](#)SUBSCRIBE
KG

5:00:06 / 5:33:30 • (Chapter-9: Transaction)- What is Transaction, ACID Properties, Transaction Sates... >



CASCADING ROLLBACK Vs CASCADELESS SCHEDULE

- It is a phenomenon, in which even if the schedule is recoverable, the a single transaction failure leads to a series of transaction rollbacks, is called cascading rollback. Cascading rollback is undesirable, since it leads to undoing of a significant amount of work. Uncommitted reads are not allowed in cascade less schedule.

- A schedule in which for each pair of transactions T_i and T_j , such that if T_j reads a data item previously written by T_i then the commit or abort of T_i must appear before read operation of T_j . Such a schedule is called cascade less schedule.

S		
T_1	T_2	T_3
R(X)		
W(X)		
	R(X)	
	W(X)	
		R(X)
C		
	C	
		C

S		
T_1	T_2	T_3
R(X)		
W(X)		
C		
	R(X)	
	W(X)	
	C	
		R(X)
		C

NON- RECOVERABLE Vs RECOVERABLE SCHEDULE

- A schedule in which for each pair of transaction T_i and T_j , such that if T_j reads a data item previously written by T_i , then the commit or abort operation of T_i appears before T_j . Such a schedule is called Non- Recoverable schedule.
- A schedule in which for each pair of transaction T_i and T_j , such that if T_j reads a data item previously written by T_i , then the commit or abort of T_i must appear before T_j . Such is called Recoverable schedule.

S	
T_1	T_2
R(X)	
W(X)	
	R(X)
	C
C	

$x = \frac{x}{y}$

S	
T_1	T_2
R(X)	
W(X)	
	R(X)
C	
	C

Two schedules S and S' are view equivalent, if they satisfy following conditions –

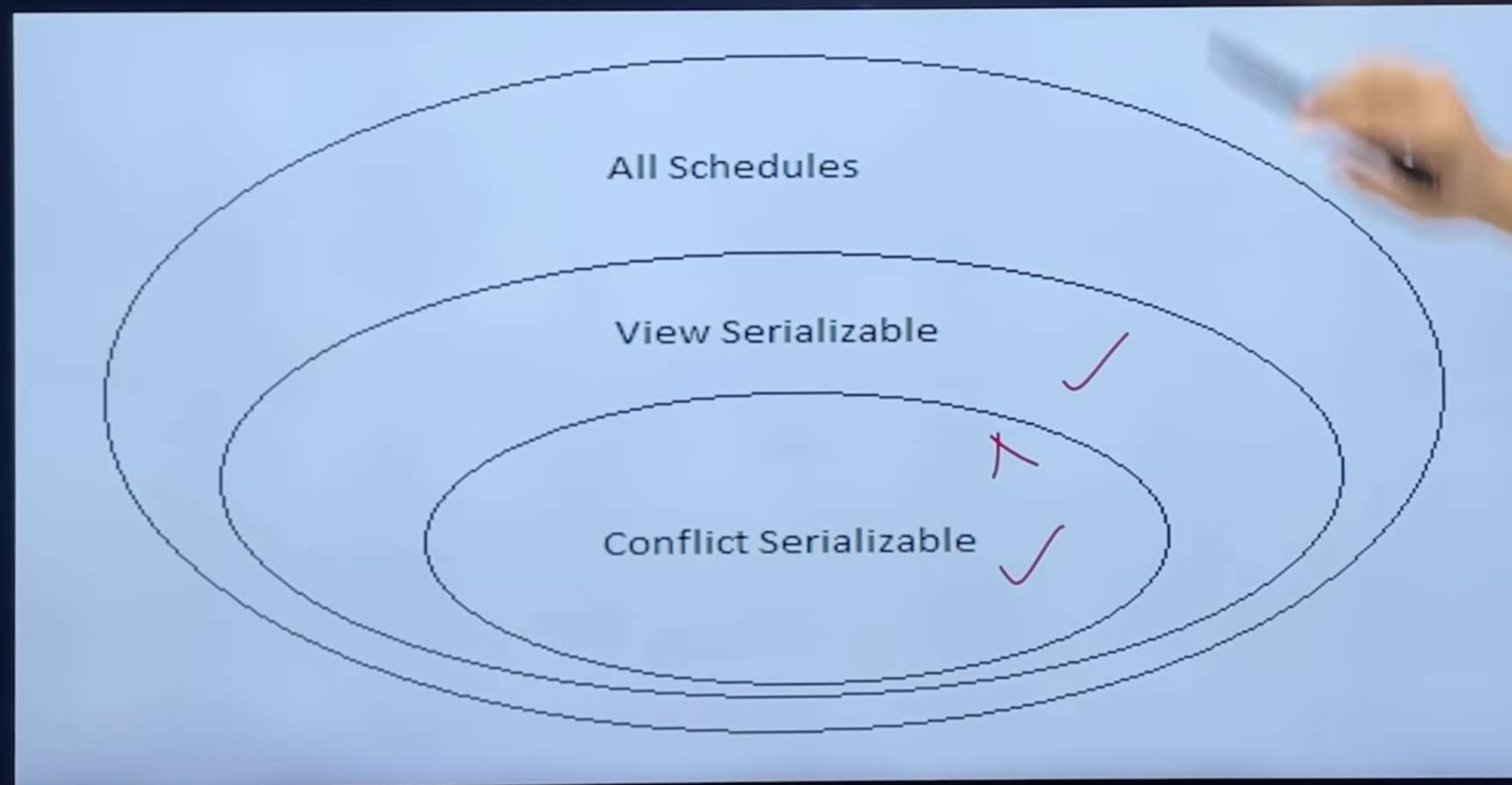
- For each data item Q , if the transaction T_i reads the initial value of Q in schedule S , then the transaction T_i must, in schedule S' , also read the initial value of Q .
- If a transaction T_i in schedule S reads any data item Q , which is updated by transaction T_j , then a transaction T_i must in schedule S' also read data item Q updated by transaction T_j in schedule S .
- For each data item Q , the transaction (if any) that performs the final write(Q) operation in schedule S , then the same transaction must also perform final write(Q) in schedule S' .

T_1	T_2
read(A) write(A)	read(A) write(A)
read(B) write(B)	read(B) write(B)

T_1	T_2
read(A) write(A) read(B) write(B)	read(A) write(A) read(B) write(B)

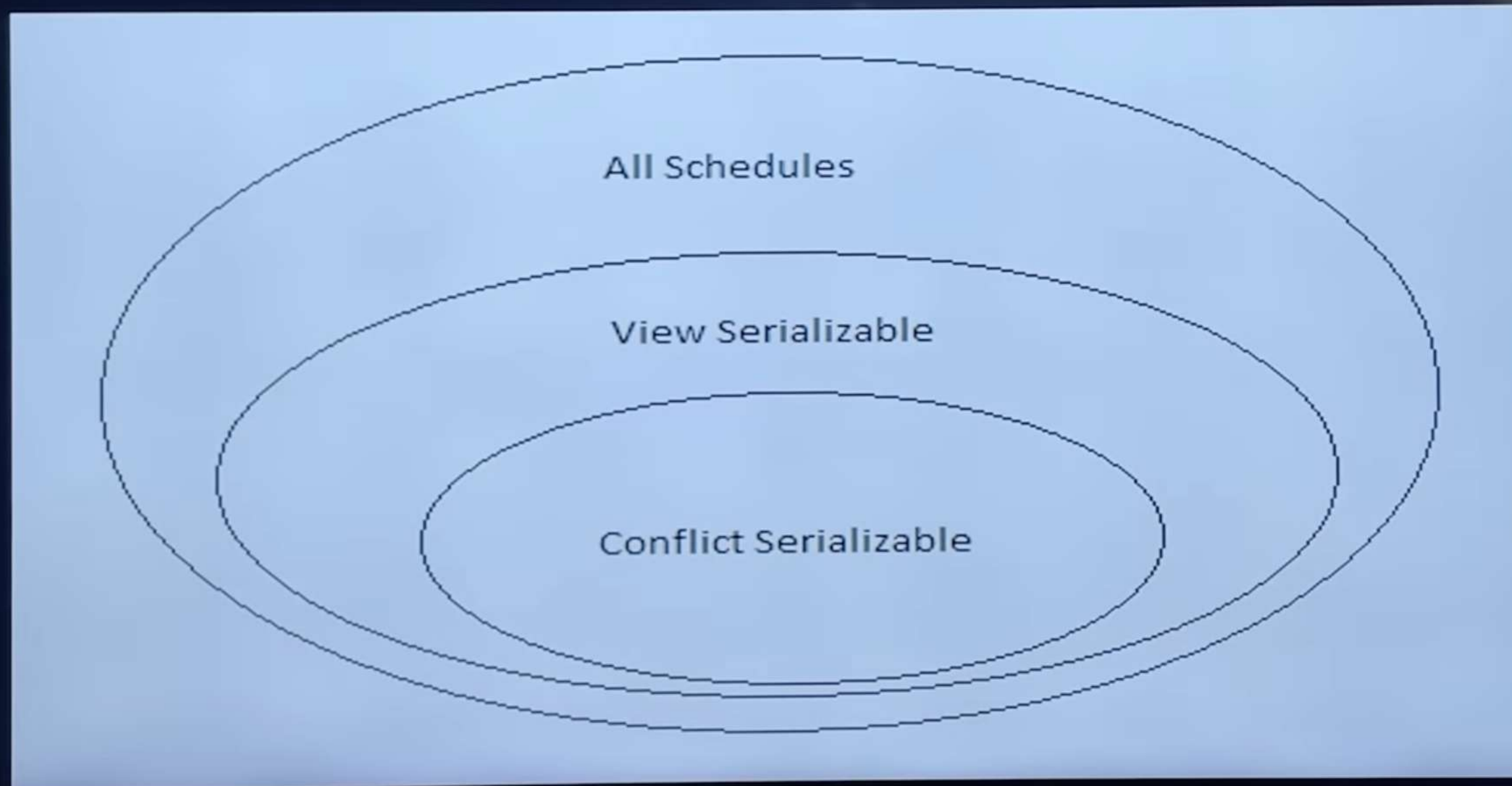
VIEW SERIALIZABLE

- If a schedule is not conflict serializable, still it can be consistent, so let us study a weaker form of serializability called View serializability, and even if a schedule is view serializable still it can be consistent.
- If a schedule is conflict serializable then it will also be view serializable, so we must check view serializability only if a schedule is not conflict serializable.



VIEW SERIALIZABLE

- If a schedule is not conflict serializable, still it can be consistent, so let us study a weaker form of serializability called View serializability, and even if a schedule is view serializable still it can be consistent.
- If a schedule is conflict serializable then it will also be view serializable, so we must check view serializability only if a schedule is not conflict serializable.

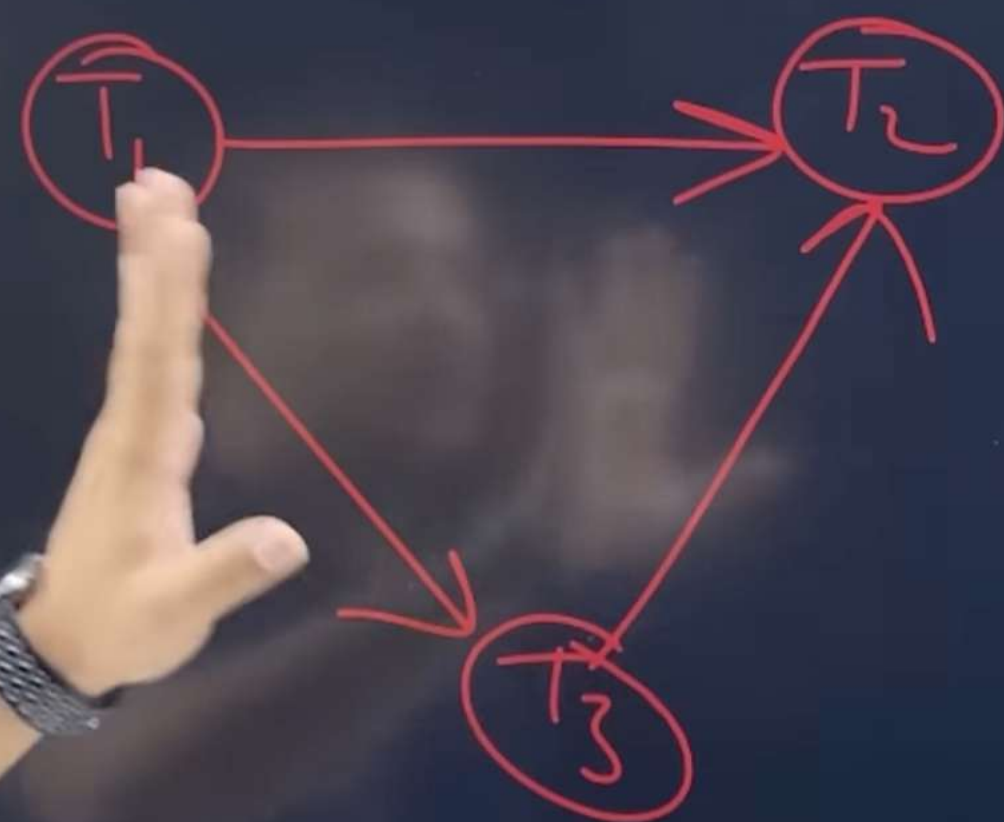


Procedure for determining conflict serializability of a schedule

- It can be determined using PRECEDENCE GRAPH method:
- A precedence graph consists of a pair $G(V, E)$
- V = set of vertices consisting of all the transactions participating in the schedule.
- E = set of edges consists of all edges $T_i \rightarrow T_j$, for which one of the following conditions holds:
 - T_i executes write(Q) before T_j executes read(Q)
 - T_i executes read(Q) before T_j executes write(Q)
 - T_i executes write(Q) before T_j executes write(Q)
- If an edge $T_i \rightarrow T_j$ exists in the precedence graph, then in any serial schedule S' equivalent to S , T_i must appear before T_j .
- **If the precedence graph for S has no cycle, then schedule S is conflict serializable, else it is not.**

Q Consider the following schedule for transactions T_1 , T_2 and T_3 : what is the correct serialization of the above?

$T_1 \rightarrow T_3 \rightarrow T_2$



<u>T1</u>	<u>T2</u>	<u>T3</u>
Read (X)	Read (Y)	Read (Y)
	Write (Y)	
Write (X)		Write (X)
	Read (X)	
	Write (X)	



CONFLICT SERIALIZABLE

- The schedules which are conflict equivalent to a serial schedule are called conflict serializable schedule.
- If a schedule S can be transformed into a schedule S' by a series of swaps of non-conflicting instructions, then S and S' are conflict equivalent. A schedule S is **conflict serializable**, if it is conflict equivalent to a serial schedule.

S

T_1	T_2
read(A) write(A)	read(A) write(A)
read(B) write(B)	read(A) write(A)
write(B)	write(B)

S'

T_1	T_2
read(A) write(A) read(B) write(B)	
	read(A) write(A) read(B) write(B)